

Exploring Data Types

Unit 1 Lesson 3

What do you need?

- Notebook for taking notes
- Open VS Code at your workstation
- Unit 1 Guide

Objective

- *I can use and describe data types like strings, integers, floats, and booleans in Python.*

Agenda

- **Intro to Data Types**
 - Strings (Slide 8)
 - Integers + Floats (Slide 11)
 - Booleans (Slide 20)
- **Checking Data Types**
- **Real-World Applications**
- **Wrap-Up and Review**

Unit 1: Python Basics

Lesson 3: Exploring Data Types

Today's Learning Goals

By the end of this lesson, you will:

- Identify and work with Python's data types:
 - Strings ("Hello").
 - Integers (42).
 - Floats (3.14).
 - Booleans (True, False).
- Choose the correct data type for specific tasks in a program.



Objective

Learning Objective:

I can use and describe Python's data types — strings, integers, floats, and booleans — and explain how they are used in simple programs.

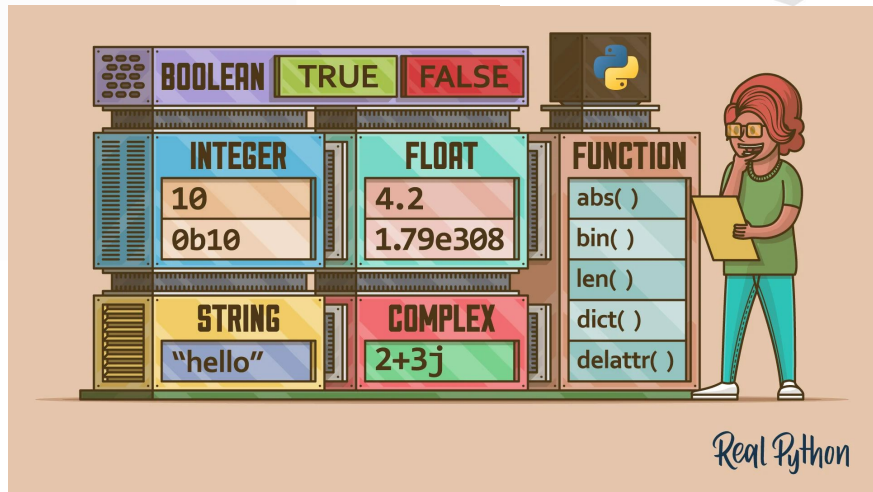
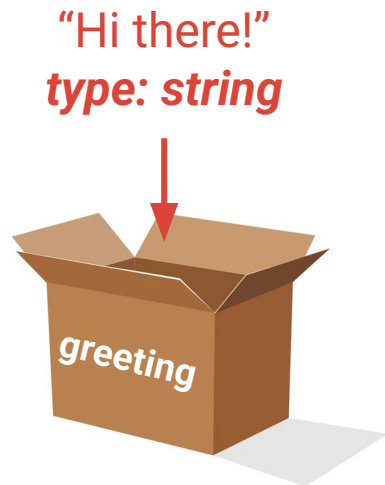
Language Objective:

I can talk and write about data types by describing what kind of information each one stores, using complete sentences and Python examples (for example, "An integer stores whole numbers like 4 or 9.").



What are Data Types?

- Data types define the kind of information a variable stores.
- **Examples of Data Types in Python:**
 - Strings: Text data
 - "Hello"
 - Integers: Whole numbers
 - 42
 - Floats: Decimal numbers
 - 3.14
 - Booleans: True/False values
 - True



Why Do Data Types Matter?

Data types determine:

- What operations you can perform.
- How data is stored in memory.

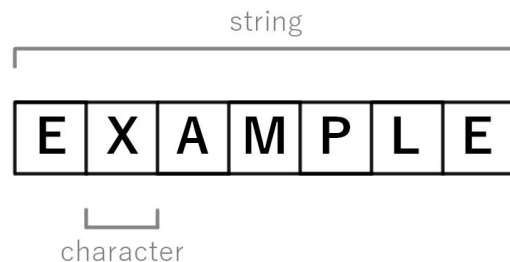
Example:

- Adding numbers: $2 + 3 \rightarrow 5$.
- Adding text: `"Hi" + " there" → "Hi there"`.



Similar to a shape sorting toy, each data type has a specific use— *it doesn't just fit anywhere!*

Strings



- What Are Strings?
 - A sequence of characters enclosed in quotes: "Hello" or 'Python'
- Key Operations:
 - **Concatenation:** Combine strings: "Hello" + " World" = "Hello World"
 - **Repetition:** Repeat strings: "Hi" * 3 = "HiHiHi"
 - **Length:** len("Hello") = 5
- Real-World Example:
 - Creating a welcome message: "Welcome, Alex!"

You Try: Use Strings

1. Write a string variable with your name.
2. Concatenate your name with a greeting.
3. Try repeating the string.

Use the previous slide to help you.

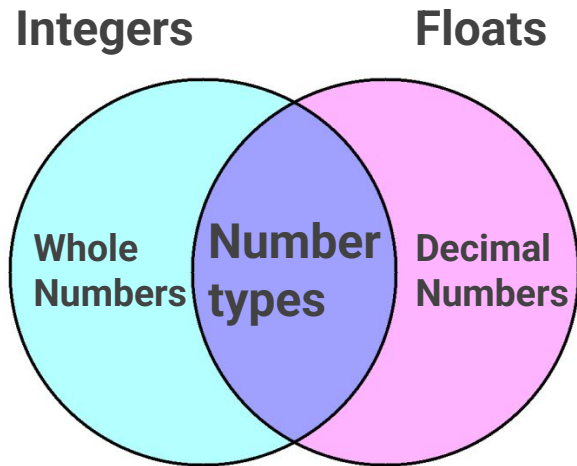
Helpful Vocab

Concatenation means **joining two or more strings together** into one single string. Think of it like connecting Lego blocks to build a longer piece or combining words to make a sentence.

In Python, you can concatenate a string and a variable that contains a string value using the `+` operator. This is helpful for creating personalized or dynamic messages.

Integers and Floats

- What Are **Integers**?
 - Whole numbers: 42, -7, 0
 - Use: Counting items, tracking scores.
- What Are **Floats**?
 - Numbers with decimals: 3.14, -0.5
 - Use: Measurements, calculations like currency.
- **Example:**
 - `num_apples = 10` (integer).
 - `price_per_apple = 0.5` (float).



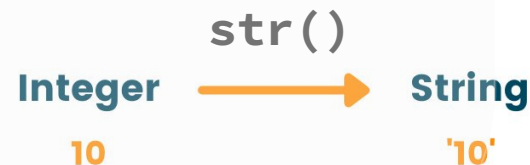
Concatenation with Non-String Values

There's a rule in Python that you cannot concatenate with a non-string value (like a number or boolean).

If a variable contains a non-string value, you'll need to convert the integer or float into a string using **str()** before concatenating.

Here's an example:

```
age = 25
message = "You are " + str(age) + " years old."
print(message)
```



print() with Commas

- There's another approach we can take besides concatenation.
 - We can use commas to separate our code within our print statement.
- Here's an example:

```
# Creating a variable  
player_score = 150 # Integer representing the player's score  
  
# Printing the variable with a label using a comma  
print("Score:", player_score) # Outputs: Score: 150
```

Why Use Commas?

- **No Conversion Needed:** Variables of different types (integer, float, boolean) can be displayed without converting them to strings.
 - We don't need to use `str()` to convert non-strings for this method.
 - The comma handles combining the text with the boolean value.
- **Automatic Formatting:** Python adds spaces between items, making the output clean and readable.

Concatenation vs Commas

Example of Concatenation

```
player_score = 150 # Integer representing the player's score
score_message = "Score: " + str(player_score) # Converting the integer to a string
print(score_message) # Output: Score: 150
```

Example of Using Commas

```
player_score = 150 # Integer representing the player's score
print("Score:", player_score) # Output: Score: 150
```

Performing Operations

- When you operate using two integers, your answer will be an integer.
 - $3 + 5 = 8$
- When you operate using two floats, your answer will be a float.
 - $3.0 + 5.0 = 8.0$
- When you operate using one integer and one float, your answer will be a float.
 - $4 * 2.5 = 10.0$

Performing Operations

- When you operate using two integers, your answer will be an integer.
 - $3 + 5 = 8$
- When you operate using two floats, your answer will be a float.
 - $3.0 + 5.0 = 8.0$
- When you operate using one integer and one float, your answer will be a float.
 - $4 * 2.5 = 10.0$

You Try: Use Integers and Floats

1. Create Variables:

- Create an integer variable to store the number of apples.
- Create a float variable to store the price per apple.

2. Calculate the Total Cost:

- Multiply the number of apples by the price per apple to get the total cost.

3. Convert the Total Cost to a String and Concatenate It:

- Use `str()` to convert the result to a string.
- Create a message to display the total cost using string concatenation.

4. **Bonus:** Use comments to label each of the 3 parts of your code!

Float Precision

- Floats may have precision issues.
- Example:
 $0.1 + 0.2 \rightarrow 0.30000000000000004$
- To clean this up, we can use **round()**, which we will cover in a later video.
- Why does this happen?
 - Computers store decimal numbers differently, causing slight inaccuracies.



Floating Point Arithmetic in Python

```
>>> .2 + .2 + .2 == .6  
False  
  
>>> .2 + .2 + .2 == .6000000000000001  
True
```

Why Use Floats Even Though They Aren't Always Precise?

1. Floats Are Ideal for Real-World Measurements

- Floats allow us to work with numbers that have decimal points, making them perfect for real-world scenarios like measuring distance, weight, or money.
- **Example:** A price of \$3.99 can't be represented as an integer because it includes cents.

Why Use Floats Even Though They Aren't Always Precise?

2. Precision is Usually Good Enough

- While floats can have minor precision errors (like $0.1 + 0.2$ resulting in 0.30000000000000004), these errors are typically very small and don't significantly impact most calculations.
- For critical use cases, like finance or engineering, we can manage this precision with **rounding**.

```
round(1.68, 1)
```

1.7

round the number i.e.
1.68 to one decimal
place

```
round(1.68)
```

2

round the number i.e.
1.68 to zero decimal
place

Why Use Floats Even Though They Aren't Always Precise?

3. Easy to Work With in Python

- Floats are simple to use in operations, and Python automatically converts between integers and floats when necessary.
- **Example:** Multiplying an integer by a float gives you a float:

```
total = 3 * 1.25 # Result: 3.75
```

Why Use Floats Even Though They Aren't Always Precise?

4. Alternative Data Types Can Be Overkill

- Python has other data types, like `Decimal` in the `decimal` library, that provide exact precision but are more complex and slower. Floats are a good balance between simplicity and functionality.
- For our purposes, we will learn to round floats but we will not need to worry too much about some of the alternatives.

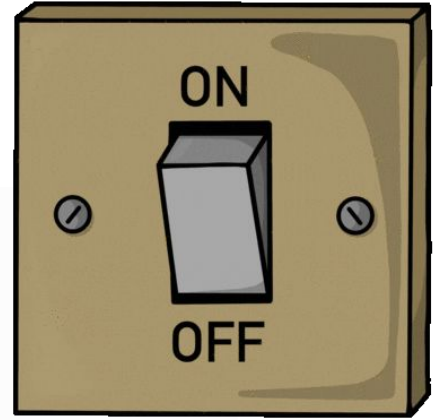
Using Floats in Our Class

- Floats are accurate enough for most real-world tasks.
 - Use the `round()` function to clean up precision issues in your output if needed:

```
result = 0.1 + 0.2  
print(round(result, 2))  # Output: 0.3
```

- When working with critical applications (like banking software), you may need specialized libraries like `decimal.Decimal` for exact precision.

Booleans



- What Are Booleans?
 - Represent yes/no answers: True, False
- Use Cases:
 - Answer yes/no questions:
 - Is the game over? → `is_game_over = True`.
 - Has the user logged in? → `is_logged_in = False`.
- Example Code:

```
if is_light_on:  
    print("The light is on!")
```

Using Booleans

- We can use booleans to check **conditions**:

```
is_logged_in = True
if is_logged_in:
    print("Welcome back!")
```

- Toggle **states**:

```
is_light_on = False
is_light_on = not is_light_on # Switch state
print(is_light_on)
```

Helpful Vocab

A **state** describes the current condition or mode of something. In programming, we often use states to represent whether something is active, inactive, on, off, or in any other defined condition. States help us keep track of what's happening in a program.

This is called a toggle switch. It switches between on/off or True/False states.



Boolean Light Bulb Example

- Imagine a light bulb with two possible conditions:
 - The light is ON.
 - The light is OFF.
- We can use a boolean variable to represent the state of the light bulb. A boolean can hold one of two values: `True` or `False`. In this example:
 - `True` means the light bulb is ON
 - `False` means the light bulb is OFF



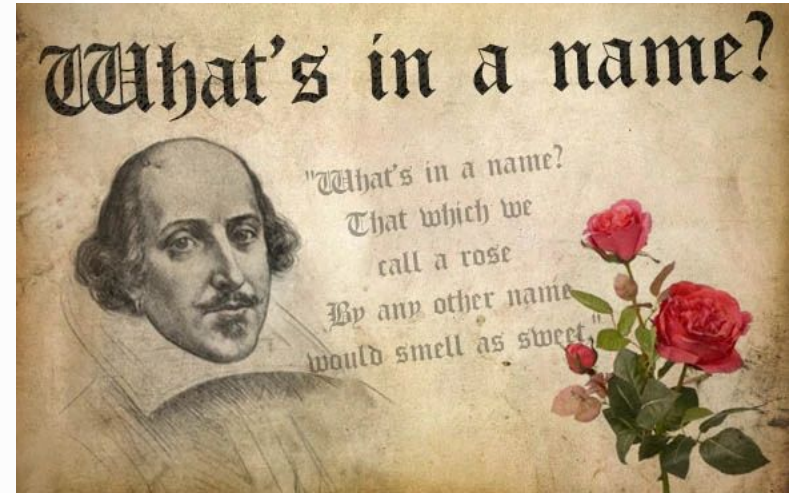
True



False

Naming Boolean Variables

- When creating boolean variables in Python, it's important to use clear and descriptive names that indicate what the variable represents. Boolean variables typically store `True` or `False` values, so their names should make it easy to understand the condition or state they describe.



1. Start with a Verb or Question Word:

- Use words like `is`, `has`, `can`, `should`, or `was` to indicate the condition being tracked.
- **Examples:**
 - `is_light_on` → Tracks whether the light is ON or OFF.
 - `has_access` → Indicates if someone has permission or not.

2. Keep It Descriptive:

- Ensure the name clearly reflects the purpose of the variable.
- **Examples:**
 - `is_game_over` → Indicates whether the game has ended.
 - `should_display_menu` → Indicates whether the menu should be shown.

3. Avoid Ambiguity:

- Use names that explicitly indicate the state or condition.
- **Bad:** status (too vague).
- **Good:** is_logged_in (clearly represents a condition).

4. Follow Python's Naming Conventions

- Use snake_case (lowercase letters with underscores) for variable names.
- Example: **is_user_active** instead of isUserActive or IsUserActive.

Examples of Good Boolean Names

- Here are some examples of well-named boolean variables in real-world scenarios:

```
is_door_locked = True          # Tracks if the door is locked
has_completed_level = False    # Tracks if the player has finished the level
can_make_purchase = True       # Indicates if a user has enough balance to buy something
should_send_email = False      # Indicates if an email needs to be sent
```

Why Naming Matters

- Using clear naming conventions for booleans makes your code easier to read, understand, and maintain. A good variable name tells you at a glance what the condition represents without needing additional comments or context.



Checking Data Types

- Use the `type()` function to check the data type of a variable.
- Example:

```
print(type("Hello"))    # Output: <class 'str'>
print(type(42))          # Output: <class 'int'>
print(type(3.14))        # Output: <class 'float'>
print(type(True))        # Output: <class 'bool'>
```

You Try: Checking Data Types

- Try checking the types of variables you created in previous tasks:

```
print(type(num_apples))  
print(type(price_per_apple))  
print(type(homework_done))
```

- **Bonus:** Create a new variable and check its type!

Why Type Conversion Matters

- In Python, some operations only work with specific data types.
- Example: You can't add a string and an integer directly!
- If you try, Python will show an error like:

```
TypeError: can only concatenate str (not "int") to str
```

Type conversion is like converting inches to centimeters so your measurements match.

Converting Between Types

```
# Converting numbers to strings
age = 16
print("I am " + str(age) + " years old.")

# Converting strings to numbers
hours = "5"
print(float(hours) * 2) # Output: 10.0
```

Function	Converts To	Example
<code>str()</code>	String	<code>str(42)</code> → <code>"42"</code>
<code>int()</code>	Integer	<code>int("7")</code> → <code>7</code>
<code>float()</code>	Float	<code>float("3.14")</code> → <code>3.14</code>

Checking Data Types with `type()`

Python lets us “check its own work.”

Using `type()` helps verify what kind of data a variable is holding.

```
name = "Maya"
age = 16
print(type(name))  # <class 'str'>
print(type(age))   # <class 'int'>
```

Real-World Type Conversion Example

You build a trip planner. The user types “5” for how many friends are going — but that’s a string!

You need to convert it to a number before doing math.

`input()` *always* needs a type conversion for non-strings.

```
friends = int(input("How many friends are coming? "))
cost_per_person = 20.0
total = friends * cost_per_person
print("Total cost:", total)
```

Choosing the Right Data Type

- Use strings for text: "Player1"
- Use integers for counts or scores: 100
- Use floats for precise numbers: 75.5
- Use booleans for conditions: True
- Example Problem:
 - Track a player's stats:
 - Name → `player_name = "AlexTheGreat"` (string)
 - Score → `player_score = 42` (integer)
 - Health → `player_health = 85.5` (float)
 - Alive → `is_alive = True` (boolean)

Practice: Putting It All Together

- Example Shopping Cart:

```
item = "Apple"      # String
quantity = 3        # Integer
price = 0.5          # Float
in_stock = True     # Boolean
total_cost = quantity * price
print("Total:", total_cost)
```

- Create your own shopping cart program using these variables.

Examples of Data Types

Category
(string)

Favorited?
True/False
(boolean)

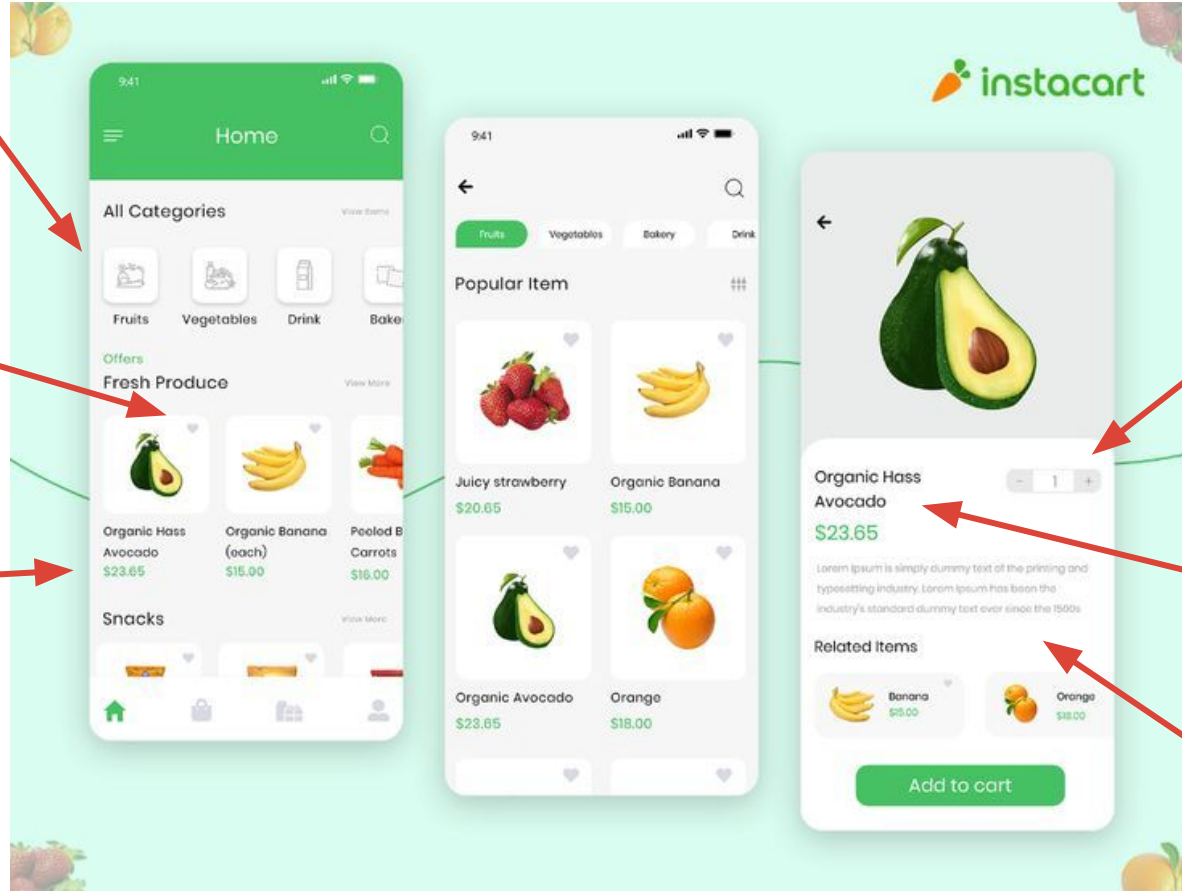
Price
(float)

In Stock?
True/False
(boolean)

Quantity
(integer)

Item name
(string)

Item
description
(string)



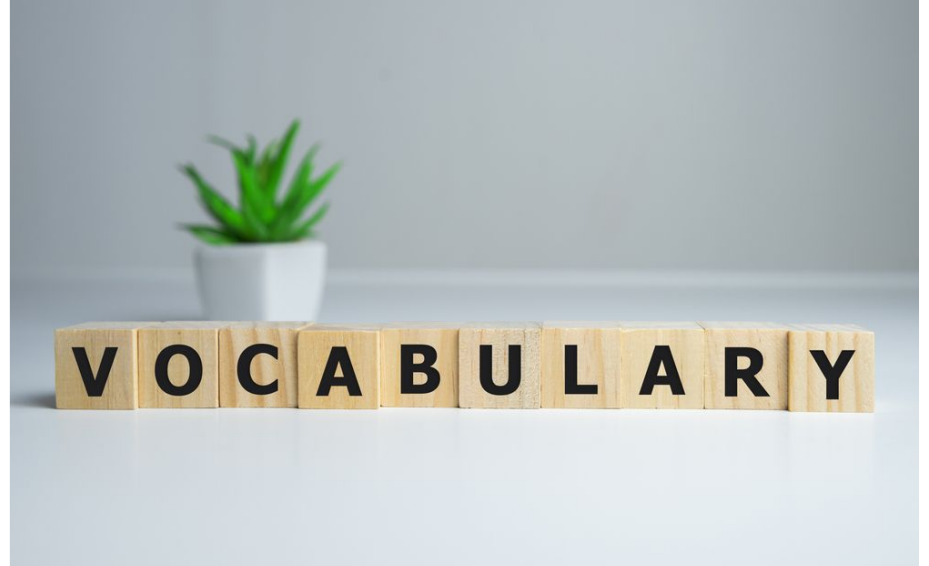
Preview — f-Strings (Coming soon!)

You don't have to use this yet — but notice how clean it looks!
Soon we'll learn how to make print statements like this.

```
# Just a preview!  
name = "Jordan"  
age = 16  
print(f"My name is {name} and I'm {age} years old.")
```

Key Vocabulary

- Data Type
- String
- Integer
- Float
- Boolean
- `type()` function



Check Your Understanding

- What are the data types of the following?

```
age = 18  
name = "Jordan"  
is_student = True  
gpa = 3.75
```

- Write your answers and then verify using `type()`.

Where to?

Video + Slides

Practice

Mastery Check

You are here!

The video and slides will include crucial information to help introduce you to concepts.

Build your skills!
Explore these concepts in depth and prepare to demonstrate mastery in the process.

Prove to yourself that you have built your skills and mastered the content!

Need Help?

Check the **Unit Guide** for additional information, including finding other resources, understanding what work is assigned, and setting yourself up for success.

